TUGAS IF5171 PEMBELAJARAN MESIN DSAI

ARIEF PURNAMA MUHARRAM - 23521013

RESUME ANN

(Mitchell, T., Machine Learning, 1997, McGraw-Hill)

Pendahuluan

Artificial Neural Network (ANN) merupakan suatu pendekatan fungsi target yang termotivasi oleh sistem biologi berupa hubungan kompleks antara jaringan saraf dalam melakukan suatu proses pembelajaran *(training)*. ANN menyediakan pendekatan fungsi target pada nilai yang bersifat *real*, diskrit, maupun vektor yang lebih *robust*. Salah satu contoh dari implementasi ANN adalah algoritma *backpropagation*.

ANN dapat digunakan untuk pendekatan pada kasus permasalahan dengan data latih yang bersifat *noisy* dan kompleks. Selain itu, ANN cocok untuk pendekatan permasalahan-permasalahan sebagai berikut.

- *Instance* direpresentasikan oleh pasang atribut-nilai yang sangat banyak.
- *Output* dari fungsi target yang diharapkan dapat berupa nilai *real*, diskrit, ataupun vektor yang terdiri atas beberapa nilai *real* ataupun diskrit.
- Data latih yang digunakan mengandung beberapa kesalahan *(error)*.
- Memungkinkan waktu *training* yang panjang, dikarenakan proses pelatihan ANN membutuhkan waktu yang cukup panjang.
- Membutuhkan evaluasi fungsi target yang dihasilkan dengan cepat.
- Kemampuan manusia untuk memahami bagaimana proses menghasilkan fungsi target tidak penting.

Perceptron

Suatu sistem ANN yang terdiri atas satu unit saja disebut sebagai *perceptron*. Sebuah *perceptron* mengambil vektor dari nilai *real* sebagai *input*, kemudian menghitung kombinasi linear dari *input* tersebut, kemudian mengeluarkan nilai 1 apabila nilai yang dihasilkan melewati ambang batas *(threshold)* tertentu dan mengeluarkan -1 untuk sebaliknya. Lebih lanjut, apabila diberikan serangkaian *input* x_1 sampai x_n maka keluaran *(output)* dari *perceptron* dapat direpresentasikan sebagai berikut.

$$O(x_1, \ldots, x_n) = \begin{cases} 1, w_0 + w_1x_1 + \ldots + w_nx_n > 0 \\ -1, w_0 + w_1x_1 + \ldots + w_nx_n < 0 \end{cases}$$

dimana setiap w_i merepresentasikan konstanta bernilai *real* (disebut juga sebagai nilai bobot *(weight)*) yang memengaruhi kontribusi nilai fitur x_i terhadap *output* dari *perceptron*.

Untuk memperingkas notasi, apabila diasumsikan nilai $x_0 = 1$ maka pertidaksamaan tersebut dapat dituliskan sebagai $\sum_{i=0}^{n} w_ix_i > 0$, atau dalam bentuk vektornya dapat dituliskan sebagai $\vec{w}_i \cdot \vec{x}_i > 0$. Dengan demikian, fungsi *perceptron* juga dapat dituliskan sebagai berikut.

$$o(\vec{x}) = sgn(\vec{w}_i \cdot \vec{x}_i)$$

dimana,

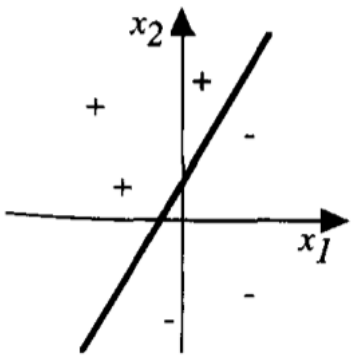
$$sgn(y) = \begin{cases} 1, y > 0 \\ -1, y < 0 \end{cases}$$

Proses *training* dengan *perceptron* melibatkan pemilihan nilai-nilai bobot $w_0, \ldots, w_n$ yang tepat. Dengan demikian, ruang kandidat hipotesis H pada pelatihan *perceptron* melibatkan set dari seluruh kemungkinan verktor bobot bernilai *real* yang dapat direpresentasikan sebagai berikut.

$$H = \{\vec{w} | \vec{w} \in \mathbb{R}^{(n+1)}\}$$

Representasi Perceptron

Suatu *perceptron* dapat dilihat sebagai suatu bidang *hyperplane* pemisah dalam suatu ruang berdimensi-n dari *instance-instance* yang terlibat. *Perceptron* memberikan *output* 1 apabila suatu *instance* terletak di satu sisi dari *hyperplane*, sedangkan -1 di sisi lainnya. *Instance-instance* yang dapat dipisahkan oleh *hyperplane* disebut sebagai *linearly separable*.



Gambar 1. Ilustrasi pemisahan kelas positif (+) dan negatif (-) yang dilakukan dengan benar oleh *perceptron* dengan *hyperplane* sebagai bidang pemisah.

Perceptron sebagai Fungsi Boolean

Sebuah *perceptron* dapat berperan sebagai fungsi *boolean*. Sebagai contoh, jika diasumsikan nilai *boolean* 1 *(true)* dan -1 *(false)*, maka salah satu cara menggunakan *two-input perceptron* sebagai fungsi AND adalah dengan menggunakan nilai $w_0 = -.8$ dan $w_1 = w_2 = .5$, sedangkan fungsi OR dapat dilakukan dengan cukup mengubah nilai $w_o = -.3$. Jika suatu *two-input perceptron* direpresentasikan dengan $o(x_1, x_2) = w_0 + w_1x_1 + w_2x_2$ maka implementasinya sebagai fungsi AND dan OR dapat direpresentasikan sebagai berikut.

AND						
x_1	x_2	w_0	w_1x_1	w_2x_2	y	$sgn(y)$
1	1	$-.8$	.5	.5	.2	1
1	0	$-.8$	.5	0	$-.3$	-1
0	1	$-.8$	0	.5	-3	-1
0	0	$-.8$	0	0	$-.8$	-1

OR						
x_1	x_2	w_0	w_1x_1	w_2x_2	y	$sgn(y)$
1	1	$-.3$	.5	.5	.7	1
1	0	$-.3$	.5	0	.2	1
0	1	$-.3$	0	.5	.2	1
0	0	$-.3$	0	0	-3	-1

Fungsi AND dan OR dapat dilihat sebagai kasus khusus ***m-of-n function***, yang berarti m dari n *input* terhadap *perceptron* harus bernilai benar. Pada fungsi OR nilai $m = 1$, sedangkan AND bernilai $m = n$. Setiap *m-of-n function* dapat direpresentasikan dengan membuat nilai bobot *input* sama (misalkan 0.5) dan mengatur nilai *threshold* w_0 sedemikian rupa sehingga dapat merepresentasikan baik fungsi AND ataupun OR.

Perceptron juga dapat digunakan untuk merepresentasikan fungsi *boolean* lainnya seperti NAND dan NOR. Akan tetapi, beberapa fungsi *boolean* tidak dapat direpresentasikan hanya dengan sebuah *perceptron*. Misal, fungsi XOR dimana nilai 1 jika dan hanya jika $x_1 \neq x_2$.

Perceptron Training Rule

Agar *perceptron* dapat melakukan klasifikasi dengan tepat maka dibutuhkan pemilihan nilai-nilai bobot yang tepat untuk *perceptron* tersebut. Oleh karena itu, diperlukan suatu algoritma untuk menentukan nilai bobot yang tepat. Terdapat dua algoritma, yaitu ***perceptron rule*** dan ***delta rule***. Pendekatan *perceptron rule* dapat digunakan jika dan hanya jika data latih yang disediakan bersifat *linearly separable* dan nilai *learning rate* (η) yang digunakan cukup kecil.

Proses pencarian nilai bobot dimulai dengan menginisialisasi nilai bobot sembarang, kemudian diterapkan pada data latih yang diberikan. Nilai bobot akan diubah apabila terjadi kesalahan klasifikasi pada data latih yang diberikan. Proses terus diulang sampai *perceptron* mampu mengklasifikasikan data latih yang diberikan dengan benar. Proses pengubahan nilai bobot w_i dilakukan berdasarkan aturan sebagai berikut.

$$w_i \leftarrow w_i + \Delta w_i$$

dimana,

$$\Delta w_i = \eta(t - o)x_i$$

Pada persamaan tersebut, t merepresentasikan target *output* dari data latih yang diberikan, o sebagai hasil prediksi *output* dari *perceptron*, dan η merupakan konstanta positif yang disebut sebagai *learning rate*. Peran dari *learning rate* yaitu mengatur derajat perubahan nilai bobot yang memungkinkan pada setiap tahapan *training*.

Gradient Descent dan Delta Rule

Apabila data yang disediakan tidak bersifat *linearly separable* maka tidak akan tercapai kondisi konvergen. Pendekatan ***delta rule*** merupakan solusi apabila data yang diberikan bersifat *non linearly separable*.

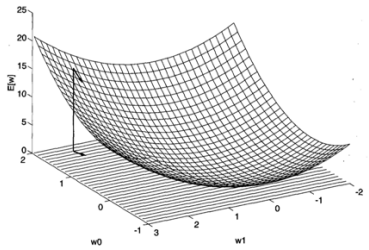
Ide dasar dari *delta rule* yaitu menggunakan pendekatan *gradient descent* untuk melakukan pencarian pada ruang hipotesis terhadap nilai vektor bobot yang memungkinkan sehingga diperoleh nilai bobot terbaik terhadap data latih yang diberikan.

Untuk memperoleh nilai bobot maka dimulai dengan melakukan pengukuran *training error* terhadap sebuah hipotesis. Pendekatan pengukuran *error* yang umum digunakan direpresentasikan sebagai berikut.

$$E(\vec{w}) \equiv \frac{1}{2}\sum_{d \in D}(t_d - o_d)^2$$

dimana D merupakan set data latih, t_d adalah target *output* dari data latih d, dan o_d merupakan prediksi *output* dari data latih d.

Proses pencarian nilai bobot yang memungkinkan berdasarkan nilai *error* pada suatu ruang hipotesis dapat divisualisasikan sebagai berikut.



Gambar 2. Visualisasi ruang hipotesis

Gradient descent bertujuan untuk mencari vektor bobot yang meminimalkan nilai *error* dengan sebuah inisialisasi nilai bobot acak, yang kemudian perlahan diubah tahapan demi tahapan kecil.

Untuk mengetahui arah perpindahan nilai pada ruang hipotesis maka perlu dilakukan operasi turunan E terhadap masing-masing komponen vektor $\vec{w}$; disebut sebagai gradien E terhadap $\vec{w}$, dituliskan sebagai $\nabla E(\vec{w})$.

$$\nabla \mathrm{E}(\vec{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right]$$

Oleh karena gradien menunjukkan arah perpindahan nilai E maka *training rule* untuk *gradient descent* menjadi,

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

dimana,

$$\Delta\vec{w} = -\eta \nabla \mathrm{E}(\vec{w})$$

Pada persamaan tersebut, η merupakan *learning rate* yang menentukan *step size* dalam proses pencarian *gradient descent*. Penggunaan tanda negatif $(-)$ dikarenakan pergerakan pencarian nilai bobot yang diharapkan menuju ke arah yang mengurangi nilai E. Lebih lanjut, *training rule* tersebut dapat direpresentasikan sebagai berikut.

$$w_i \leftarrow w_i + \Delta w_i$$

dimana,

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

Untuk merekonstruksi algoritma yang bertujuan untuk mengubah nilai bobot berdasarkan persamaan di atas maka perlu dilakukan penguraian persamaan sebagai berikut.

$$\frac{\partial E}{\partial w_i} = \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$\frac{\partial E}{\partial w_i} = \sum\nolimits_{d \in D} (t_d - o_d)(-x_{id})$$

dimana x_{id} menunjukkan *input* tunggal dari komponen x_i pada data latih d. Setelah mendapatkan penguraian $\frac{\partial E}{\partial w_i}$ maka untuk memperoleh Δw_i didapatkan persamaan sebagai berikut.

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id}$$

Berdasarkan persamaan-persamaan dan tahapan-tahapan tersebut maka algoritma *gradient descent* dapat dituliskan sebagai berikut.

GRADIENT-DESCENT(*training_examples*, η)

Each training example is a pair of the form $(\vec{x}, t)$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $(\vec{x}, t)$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i \quad \text{(T4.1)}$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i \quad \text{(T4.2)}$$

Gambar 3. Algoritma *gradient descent*

Bidang *error* dalam *gradient descent* hanya memiliki satu nilai *global minimum* sehingga pada akhirnya algoritma akan konvergen pada satu titik dengan *minimum error*. Dengan demikian, penggunaan η harus sedemikian kecil untuk menghindari *overstepping*.

Stochastic Gradient Descent

Dua permasalahan utama yang terdapat pada penggunaan algoritma *gradient descent* yaitu (1) konvergen ke suatu *local minimum* mungkin bisa berjalan sangat lambat; dan (2) jika terdapat *multiple local minima* maka tidak ada jaminan bahwa algoritma dapat menemukan *global minimum*. Untuk menyelesaikan permasalahan tersebut maka dapat digunakan pendekatan *stochastic gradient descent*.

Ide dasar dari pendekatan tersebut yaitu dengan melakukan perkiraan *(approximation)* pencarian *gradient descent* dengan memperbarui nilai

bobotnya secara bertahap *(incrementally)*, diikuti dengan perhitungan *error* untuk setiap data latih. Pendekatan *stochastic gradient descent* dilakukan dengan memperbarui persamaan *gradient descent* dengan persamaan sebagai berikut.

$$\Delta w_i = \eta(t - o)x_i$$

dimana t, o, dan x_i merupakan nilai target, prediksi, dan *input* ke- i dari data latih yang digunakan. Perhitungan *error* yang digunakan menggunakan pendekatan sebagai berikut.

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2$$

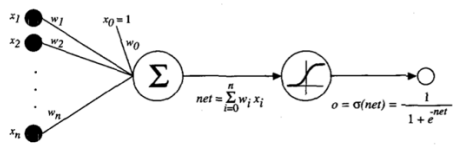
dimana t_d dan o_d merupakan nilai target dan prediksi dari data latih d. *Stochastic gradient descent* melakukan iterasi terhadap seluruh data latih d dalam D, pada setiap iterasinya memperbarui nilai bobot berdasarkan gradien terhadap $E_d(\vec{w})$.

Perbedaan terhadap *gradient descent* standar adalah sebagai berikut.

- *Stochastic gradient descent* memperbarui nilai bobot setiap data latih, sedangkan pada *gradient descent* diperlukan perhitungan nilai *error* terhadap seluruh data latih sebelum memperbarui nilai bobot.
- *Gradient descent* standar memerlukan komputasi lebih banyak setiap memperbarui nilai bobot.
- *Stochastic gradient descent* dapat menghindari *local minima* karena menggunakan pendekatan $\nabla E_d(\vec{w})$ daripada $\nabla E(\vec{w})$.

Multilayer Network

Pada banyak kasus, *multilayer network* dapat merepresentasikan *nonlinear decision surface* yang jauh lebih ekspresif daripada yang dapat dilakukan dengan *linear decision surface* dari suatu unit *perceptron* tunggal. Untuk menyusun *multilayer network* dibutuhkan satu unit tambahan yang disebut sebagai *sigmoid unit*.



Gambar 4. *Sigmoid unit*

Unit sigmoid melakukan perhitungan kombinasi linear terhadap seluruh *input*, kemudian menerapkan *threshold* terhadap hasil yang dihasilkan. Unit sigmoid dapat direpresentasikan sebagai berikut.

$$o = \sigma(\vec{w} \cdot \vec{x})$$

dimana,

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

dengan σ disebut sebagai fungsi sigmoid (atau fungsi logistik). Oleh karena perannya memetakan berbagai nilai *input* ke dalam rentang yang lebih kecil, fungsi ini juga disebut sebagai ***squashing function***. Selain fungsi sigmoid, fungsi $tanh$ juga dapat digunakan pada bagian ini.

Algoritma Backpropagation

Algoritma *backpropagation* berperan melakukan pencarian bobot untuk *multilayer network*. Algoritma tersebut memanfaatkan *gradient descent* untuk meminimalkan *squared error* antara prediksi *network* dengan nilai targetnya. Oleh karena *multilayer network* terdiri atas beberapa *output unit* maka persamaan E yang digunakan menjadi,

$$E(\vec{w}) = \frac{1}{2}\sum_{d \in D}\sum_{k \in outputs}(t_{kd} - o_{kd})^2$$

Algoritma *backpropagation* ditunjukan pada tabel sebagai berikut.

BACKPROPAGATION(*training_examples*, η, n_{in}, n_{out}, n_{hidden})

Each training example is a pair of the form $(\vec{x}, \vec{t})$, *where* $\vec{x}$ *is the vector of network input values, and* $\vec{t}$ *is the vector of target network output values.*

η *is the learning rate (e.g., .05).* n_{in} *is the number of network inputs,* n_{hidden} *the number of units in the hidden layer, and* n_{out} *the number of output units.*

The input from unit i into unit j is denoted x_{ji}, *and the weight from unit i to unit j is denoted* w_{ji}.

- Create a feed-forward network with n_{in} inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers (e.g., between −.05 and .05).
- Until the termination condition is met, Do
  - For each $(\vec{x}, \vec{t})$ in *training_examples*, Do

    Propagate the input forward through the network:

    1. Input the instance $\vec{x}$ to the network and compute the output o_u of every unit u in the network.

    Propagate the errors backward through the network:

    2. For each network output unit k, calculate its error term δ_k

    $$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \quad \text{(T4.3)}$$

    3. For each hidden unit h, calculate its error term δ_h

    $$\delta_h \leftarrow o_h(1 - o_h)\sum_{k \in outputs} w_{kh}\delta_k \quad \text{(T4.4)}$$

    4. Update each network weight w_{ji}

    $$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

    where

    $$\Delta w_{ji} = \eta\,\delta_j\,x_{ji} \quad \text{(T4.5)}$$

Gambar 5. Algoritma *backpropagation*

Penggunaan ***gradient descent weight-update rule*** serupa dengan *delta training rule*. Akan

tetapi, representasi *error* $(t - o)$ pada *delta rule* diganti dengan *error term* yang lebih kompleks, yaitu δ_j.

Momentum

Salah satu variasi umum dari pengembangan algoritma *backpropagation* yaitu dengan membuat algoritma memperbarui nilai bobot pada iterasi ke-n hanya bergantung parsial terhadap pembaruan nilai yang terjadi pada iterasi ke-$(n-1)$. Hal tersebut dapat direpresentasikan sebagai berikut.

$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1)$$

Pada persamaan tersebut $\Delta w_{ji}(n)$ melakukan perubahan nilai bobot selama n iterasi pada *loop* utama, dan nilai $0 \leq \alpha < 1$ merupakan konstanta yang disebut sebagai **momentum**. Tujuan dari efek momentum adalah memastikan bahwa arah perubahan nilai berada pada arah yang sama dari satu iterasi ke iterasi berikutnya.

Arbitrary Acyclic Network

Algoritma *backpropagation* dapat digeneralisasi menjadi *feedforward network* yang memiliki kedalaman *(depth)* tertentu dan bersifat asiklik. Oleh karena itu, diperlukan sedikit modifikasi pada perhitungan nilai δ. Secara umum, perhitungan nilai δ_r untuk unit r pada *layer* m dihitung berdasarkan nilai δ pada *layer* $m+1$ sehingga dapat direpresentasikan sebagai berikut.

$$\delta_r = o_r(1 - o_r) \sum_{s \in layer\ m+1} w_{sr} \delta_s$$

Lebih umum, persamaan tersebut dapat digeneralisasi sebagai berikut.

$$\delta_r = o_r(1 - o_r) \sum_{s \in Downstream(r)} w_{sr} \delta_s$$

dimana $Downstream(r)$ merupakan set unit r yang berada di hilir *(downstream)* dari unit r dalam *network*; yaitu seluruh unit yang menggunakan *output* dari unit r sebagai *input*-nya.

Power of FeedForward Network

Secara umum, *feedforward network* memberikan hasil yang baik pada kasus seperti representasi fungsi *boolean*, fungsi kontinu, dan fungsi *arbitrary*.

Overfitting and Stopping Criterion

Salah satu pendekatan yang jelas kapan proses pengubahan nilai bobot perlu dihentikan yaitu ketika nilai *error* E pada data latih telah memenuhi *threshold* yang diharapkan. Akan tetapi, pendekatan seperti itu merupakan strategi yang kurang baik karena algoritma *backpropagation* rentan terjadi *overfitting* terhadap data latih yang diberikan sehingga dapat menyebabkan berkurangnya akurasi generalisasi terhadap data yang *unseen*.

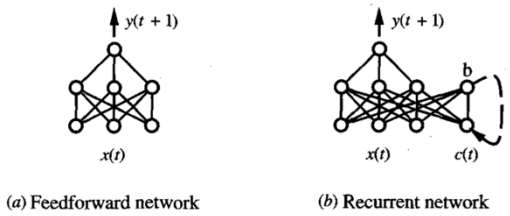
Apabila terjadi *overfitting*, algoritma *backpropagation* cenderung akan membentuk *decision surface* yang bersifat kompleks tetapi *fit* terhadap data latih. Akan tetapi, kinerja akurasi akan turun terhadap data *unseen*. Permasalahan ini serupa yang terjadi pada algoritma *decision tree learning.*

Beberapa pendekatan yang dapat dilakukan untuk mengatasi *overfitting* adalah sebagai berikut.

- ***weight decay***, yaitu mengurangi bobot oleh sejumlah faktor tertentu di setiap iterasinya.
- Menggunakan **set data validasi** dan menghentikan proses *training* pada iterasi yang menghasilkan nilai *error* terkecil pada set data validasi; menyimpan nilai bobotnya sebagai hipotesis final.
- Apabila jumlah data yang digunakan "sedikit" maka dapat menggunakan pendekatan dengan ***k-fold cross validation***.

Recurrent Network

Apabila persoalan yang akan diselesaikan melibatkan data bertipe *time series* maka dapat menggunakan ***recurrent network.*** *Recurrent network* merupakan ANN yang menggunakan data *time series* dan menggunakan *output* dari unit *network* pada waktu t sebagai *input* terhadap unit lain pada waktu $t+1$.



Gambar 6. Perbandingan (a) *FeedForward network*; (b) *Recurrent network*